

로그 RCA 증거 보존형 KV Cache 압축

구현 실행 로드맵 — 확정본

2026. 07

핵심 원칙 프로젝트를 죽일 수 있는 관문 실험을 앞으로 당긴다. 안 되는 걸 5개월 하고 깨닫는 사태를 막는다.
관문 3개(킬스위치 · PoC · Nezha)를 GO/NO-GO 결정점으로 두고, 1층 평가 완주를 '발표 가능 최소선'으로 삼아 안전판을 확보한다.

용어 풀이 — 어려운 말 쉽게

본문에 나오는 기술 용어를 한 줄로 풀었다. 읽다가 막히면 여기로 돌아와 확인하면 된다.

① 프로젝트가 무엇을 하는가

용어	쉬운 뜻
RCA (근본원인 분석)	장애가 났을 때 '진짜 원인'이 무엇인지 찾아내는 일
원인 라인	장애의 실제 원인이 적힌 로그 한 줄
KV 캐시	LLM이 긴 글을 읽을 때 앞부분 계산 결과를 저장해 두는 임시 메모리. 글이 길수록 커져서 GPU 메모리를 잡아먹는다
KV 압축	그 임시 메모리에서 덜 중요한 부분을 버려 메모리를 줄이는 것
보존율	압축하고도 '원인 라인'이 안 버려지고 남아 있는 비율. 높을수록 좋다
LogPress (우리가 만드는 것)	로그의 특성(레벨·회귀도·시간·위치)을 이용해 원인 라인은 남기고 잡음은 버리는 로그 전용 압축기

② 압축 · 모델 관련

용어	쉬운 뜻
kvpress	NVIDIA가 만든 KV 압축 '도구 모음'. 기존 압축법들이 이미 들어 있어 갖다 쓰면 된다
baseline (기준선)	우리 방법과 비교할 기준 방법들 (아래 4종)
Random·StreamingLLM·H2O·PyramidKV	기존 KV 압축법 4종. 각각 무작위로 / 최근 것 위주로 / 모델이 주목한 것 위주로 남긴다 — 희귀한 원인 라인을 잘 놓친다
토큰	모델이 글을 잘게 쪼갠 최소 단위(대략 단어 조각)
레이어 (층)	모델을 이루는 계산 단계. 입력이 여러 층을 차례로 통과한다
lost-in-the-middle	긴 글의 '중간'에 있는 정보를 모델이 잘 놓치는 현상
초기 레이어 후 적용 (FastKV식)	앞쪽 층들은 글 전체를 다 보게 두고, 그 다음 층부터 압축하는 방식. 중간 정보 놓침을 줄이려는 것
마스크	'이 줄은 버리고 이 줄은 남겨라'를 표시한 표
토큰-라인 매핑	각 토큰이 로그의 '몇 번째 줄'에서 왔는지 연결한 표
Drain3	로그에서 공통 형태(템플릿)를 뽑아 얼마나 드문지 세어 주는 도구
backbone	실험에 쓰는 기반 LLM 모델 (Llama-3.1-8B 등)

③ 실험 · 평가 관련

용어	쉬운 뜻
킬스위치	'이게 안 되면 프로젝트 접는다'는 초반 생사 판정 실험
관문 (GO / NO-GO)	통과 못 하면 다음으로 못 넘어가는 결정 지점
하드 케이스	찾기 어려운 원인들. ERROR가 아닌 원인 / 로그 중간에 묻힌 원인
충화	전체 평균만 보지 않고, 어려운 케이스만 따로 떼어 성적을 확인하는 것

용어	쉬운 뜻
1층 / 2층 평가	1층 = 원인이 얼마나 남았나(보존율). 2층 = 그래서 실제 RCA 정답률이 오르나
ablation (제거 실험)	구성 요소를 하나씩 빼 보며 각자가 결과에 얼마나 기여하는지 확인하는 실험
delta (차이)	압축 방식만 바꿨을 때 정답률이 얼마나 달라지는지 그 차이값
floor (바닥)	문제가 너무 어려워 어떤 방법이든 다 0점 근처라 우열이 안 갈리는 상태
제약형 태스크	자유서술 대신 '보기 중 고르기'처럼 답을 좁혀 floor를 피하는 문제 형식
held-out (홀드아웃)	튜닝엔 안 쓰고 최종 채점에만 쓰려고 떼어 둔 데이터
라벨 누수	정답을 미리 훑쳐봐서 성적이 부풀려지는 것
Thunderbird·BGL·Nezha	로그 공개 데이터셋. 앞 둘은 '정상/이상'만, Nezha는 '원인'까지 표시돼 있다

1. 3대 관문 · 임계 경로 · 타임라인

진짜 관문 3개 — 이 순서로 막히면 뒤가 다 무의미

관문	주차	GO 조건	담당
① 킬스위치	3주	하드케이스 표본 충분 + 신호 구별력	D
② kvpress PoC	9주	마스크 → 전 레이어 KV 삭제 동작	C (전담)
③ Nezha 전처리	11주	원인 라벨을 라인 단위로 매핑 가능	B

임계 경로: 0단계 → ①킬스위치 → 2단계 매핑 → ②PoC → 3단계 LogPress → 4단계 1층 평가. Baseline 재현과 Nezha 전처리는 병렬이라 임계 경로 밖(단 Nezha는 11주 말까지 완료 필수).

단계 ↔ 마일스톤 매핑 (번호 혼동 방지)

단계 번호(0~6)와 마일스톤 코드(M0~M7)를 함께 쓴다. 특히 '1층 평가'가 4단계이자 M5라 혼동이 잦으니 아래로 대조한다.

단계(로드맵)	마일스톤	내용	주차	유형
0단계	M0·M1	환경·데이터 + 킬스위치	1~3주	△ 관문(M1)
1단계	M2 (+Nezha 병렬)	Baseline 재현 + Nezha 착수	3~6주	○ 병렬
(병렬)	M-N	Nezha 전처리 완료	~11주	△ 관문
2단계	M3	라인→토큰 매핑 · PoC	5~9주	△ 관문
3단계	M4	LogPress 구현	8~11주	★ 핵심
4단계	M5	1층 평가 (보존율 곡선)	10~13주	★ 안전판
5단계	M6	2층 RCA delta	12~15주	★ 핵심
6단계	M7	마무리 · 정리	13~16주	✓ 마무리

타임라인 (16주) — 진행 구간과 담당 트랙

단계 w 주	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	담당
0 환경·데이터																	A·B
◆ M1 킬스위치			◆														D
1 Baseline 재현																	A
… Nezha(병렬)											◆						B
2 매핑·PoC									◆								C
3 LogPress																	C
4 1층 평가													★				D
5 2층 delta																	D
6 마무리																	D

범례: ■ 활성 ◆ 관문 ★ 안전판/핵심 담당 = 주담당 트랙(A 인프라 · B 데이터 · C 매핑/압축 · D 리드/평가)

2. 인력 — 4트랙 (A · B · C · D)

실력 대등 전제에서 트랙은 '어느 관문을 책임지는가'로 나뉜다. 각 트랙이 관문을 하나씩 진다.

트랙	한 줄 정의	취는 관문	부하 시점
A · 인프라	환경·도구·버전핀·모델 로드 + Baseline 재현	환경 지연(전원 대기)	초반 집중
B · 데이터	로그 파싱·라벨·하드케이스 카운트·Nezha 전처리	Nezha (11주)	꾸준 · 후반
C · 매핑/압축	토큰·라인 매핑·kvpress 혹은 LogPress	PoC (9주)	중반 최대
D · 리드/평가	킬스위치·보존율·delta·조율·판정	킬스위치 (3주)	처음과 끝

트랙 A — 인프라 / 환경 + Baseline 재현

- 담당 범위: RunPod/Vast.ai 세팅·종료 관리 · kvpress+transformers 버전 핀 고정 · Llama-3.1-8B/Qwen2.5-7B 로드·KV 접근 검증 · GPU 메모리 프로파일링 · Baseline 4종 (Random·StreamingLLM·H2O·PyramidKV) 재현 주담당
- 필요 역량: Linux CLI·Docker·의존성 관리·HF transformers (필수) / CUDA·클라우드 GPU (우대)
- 산출물: 재현 환경·버전 잠금 파일 + baseline 보존율 곡선(기준선)
- 관문 리스크: 0~1주 환경 지연 = 전원 대기 → 초반 최우선. kvpress↔transformers 호환성이 첫 지뢰(0단계 첫날 조사)

트랙 B — 데이터 / 라벨 + Nezha

- 담당 범위: Thunderbird/BGL 파싱·윈도우 · 하드 케이스 개수 조기 측정 · held-out 분리 · Nezha 전처리(11주 완료, D 핸드오프)
- 필요 역량: 데이터 파싱·정제·꼼꼼함(라벨이 채점 구간)·로그 포맷 (필수) / Drain3 (우대)
- 산출물: 정제 데이터셋·로더 + 하드 케이스 통계표 + Nezha 원인 라벨 평가셋
- 관문 리스크: Nezha가 5단계 생사 관문 → 3주차부터 병렬 착수 필수. 실수는 티 없이 결과 전체를 오염 → 방치 금지, D가 주 1회 점검

트랙 C — 매핑 / 압축기 (기술 핵심)

- 담당 범위: 토큰·라인 매핑(BPE 예외 처리) · 라인 특징 추출(레벨·Drain3 회귀도·위치) · kvpress 혹은 PoC · LogPress 구현('판단 한 번 / 적용 초기 레이어 후')
- 필요 역량: 트랜스포머 내부(KV layer·head 저장)·토큰나이저·남의 코드 확장 (필수) / forward hook·Cache 클래스 (우대)
- 산출물: 매핑 단위 테스트 + PoC 데모 + LogPress 클래스
- 관문 리스크: 관문 PoC가 여기 — 구현 가능성 자체를 가름. 처음부터 끝까지 가장 무거워 전담 필수(겸직 금지)

트랙 D — 리드 / 평가

- 담당 범위: 킬스위치 실행·판정 · 보존율 채점+충화(전체/비·ERROR/중간위치) · 1층 곡선·2층 delta(제약형 +floor 대응) · 통계 검정 · Ablation 3종 설계 · 전체 조율·관문 판정
- 필요 역량: 실험 설계·통계(검정력)·전체 그림·결과 프레이밍 (필수) / 평가 하네스·시각화·논문 정리 (우대)
- 산출물: 킬스위치 결론 + 충화 보존율 곡선 + RCA delta + Ablation + 최종 발표/논문
- 관문 리스크: '보존율↑가 RCA↑를 보장 안 함'을 이해하고 정직하게 프레이밍. D가 그림을 못 쥐면 트랙이 따로 돌아 판정이 흐려짐

겸직 규칙 (4명 미만 시): A+D 겸직 가능(초반·후반으로 시기가 갈림) 또는 B가 A 흡수. **B+C 겸직은 금지** — 5~11주차에 두 작업이 정면 충돌해 Nezha가 희생되고 5단계가 붕괴한다. C만은 절대 겸직 금지(관문 M3 난도).

3. 단계별 통합 상세

각 단계에 로드맵(할 일) · 마일스톤(주차·완료판정·관문) · 인력(주담당)을 한 블록으로 합쳤다.

0단계 (M0·M1) 환경 · 데이터 · 킬스위치 실험 · 1~3주차 △ 관문

환경 세팅 [A]

- RunPod에 kvpress + transformers 설치 — 버전 핀 고정 필수
- kvpress 브레이킹 체인지 이력 있음(HF transformers KV 캐싱 구조 변경). transformers 버전 먼저 확인 후 고정
- Llama-3.1-8B-Instruct 로드, past_key_values shape 확인, KV 접근 동작 검증

데이터 [B]

- Thunderbird(또는 BGL) 로드, 라인 단위 이상 라벨 파싱 · 장애 케이스 단위 로그 윈도우 자르기
- 레벨 신호 정의: Thunderbird에는 명시적 INFO/ERROR 레벨이 약하므로 error·fail·exception 등 키워드 정규식으로 정의 확정(킬스위치의 전제)

하드 케이스 조기 측정 [B]

4단계 총화 결과가 통계로 무너지는 걸 0단계에서 차단한다. 라인 라벨·레벨·위치만 있으면 지금 바로 셀 수 있다.

- 전체 원인(이상) 라인 개수
- 그중 비-ERROR(INFO/WARN) 원인 라인 개수 → 총화 (b)
- 그중 문맥 중간 위치(25~75% 구간) 원인 라인 개수 → 총화 (c)
- 케이스(장애 윈도우)당 원인 라인 분포 (0개 / 1개 / 다수)

킬스위치 실험 [D]

- 레벨 필터가 원인 라인을 몇 % 보존하는지 측정
- 필터가 놓친 원인을 회귀도·위치 신호로 회수 가능한 비율 추정(예비 검정)

구분	① 킬스위치 GO 조건
주 조건 ①	하드 케이스 표본이 통계 가능 규모 (케이스 기준 수백 개 이상)
주 조건 ②	필터가 놓친 원인이 회귀도·위치 신호로 무작위보다 유의하게 구별됨 (예비 검정)
보조 확인	레벨 필터 보존율 < 99% (상식적으로 거의 항상 만족 — 참고용)
NO-GO 대응	표본 과소 → BGL 병합 / 총화 완화 / 라인 단위 검정. 신호 구별력 없음 → 신호 재설계 또는 주제 재설정 회의

왜 99%를 내렸나: 설정 변경·배포는 INFO로, 리소스 경고는 WARN으로 찍힌다. 원인이 구조적으로 ERROR가 아닌 경우가 많아 99%는 상식적으로 나오기 어렵다 → '99% 미만'은 거의 항상 통과해 관문 역할을 못 한다. 진짜 위험은 '비-ERROR 원인이 30개뿐'인 상황인데, 이건 99% 기준으로 걸러지지 않는다.

1단계 (M2 + Nezha 병렬) Baseline 재현 + Nezha 착수 · 3~6주차 ◐ 병렬

Baseline 재현 [A — 주담당]

- kvpress로 Random · StreamingLLM · H2O · PyramidKV를 Thunderbird 윈도우에 적용
- 압축률 50~90% 스윙, baseline 보존율 곡선 기준선 확보 → 우리 목표 수치 앵커
- Qwen2.5-7B도 같은 환경에서 동작 확인(2차 backbone 초기 세팅)

Nezha 전처리 병렬 착수 [B]

- 3주차 착수 → 11주차 말 완료(총 8주 확보)
- 데이터 구조 파악, 구문 → 줄 매핑 가능 여부 탐색, 라벨 구조 검수
- 핸드오프: 11주차 말까지 B → D 인계 필수. D가 12주차 5단계를 시작하려면 이 시점에 평가셋이 손에 있어야 함
- D가 주 1회 진행 점검 — '방치 금지'(5단계 생사 관문)

사전 준비 [C · D]

- [C] kvpress 소스 사전 조사 — BasePress·ScorerPress 구조 파악, 토큰라이저 offsets_mapping 실험
- [C] 백업 경로 병렬 조사 착수(A와 함께) — transformers Cache 서브클래싱이 며칠 규모인지 파악
- [D] 보존율 채점 함수 사전 구현 + 제약형 RCA 태스크 설계(5단계 floor 회피용)

통과 조건: baseline 4종이 kvpress에서 정상 재현되고 압축률-보존율 곡선이 그려짐. Nezha 구조 파악 진척 확인.

2단계 (M3) 라인 → 토큰 매핑 인프라 (기술 핵심) · 5~9주차 △ 관문

프로젝트에서 가장 어려운 구현 다리. 여기가 진짜 연구 기여 지점. [C 전담 · 겸직 금지]

토큰-라인 매핑

- 로그 토큰화 시 각 라인의 토큰 범위 [start, end] 기록 → token_idx → line_idx 테이블 생성 (쉽게: 어느 토큰이 로그 몇 번째 줄에서 왔는지 연결하는 표를 만든다)
- BPE 토큰라이저 특성상 UUID·숫자·경로에서 분절 패턴이 가변적 → 예외 처리 필수
- offsets_mapping 활용 — HF 토큰라이저가 '이 토큰은 원문 47~52번째 글자'를 알려줌

라인별 특징 추출

- 레벨: 정규식(0단계에서 확정한 키워드 기준) · 템플릿 희귀도: Drain3 파싱 → 템플릿 빈도 역산 · 위치: 라인 순서 / 타임스탬프

① 레이어 적용 시점 (최우선)

판단은 한 번, 적용은 초기 레이어 이후.

라인 keep/drop 판단은 전 레이어 공통으로 한 번 계산한다. 단 실제 KV 삭제(적용)는 초기 레이어가 전체 시퀀스를 처리한 뒤부터 수행한다(FastKV식). 즉 '마스크 한 번 계산'과 '레이어 0부터 삭제'는 다르며, 우리는 전자만 취한다.

왜 중요한가: 레이어 0부터 무조건 지우면 '초기 레이어가 전체 문맥을 먼저 처리한다'는 이점이 사라져 \$2.1③(lost-in-the-middle 완화) 논거가 무너진다. 우리가 필터를 비판한 바로 그 구조가 되어버린다 — 도구 정당성의 핵심 축이 자기 구현과 모순되는 것이다.

kvpress 혹은 PoC (이 단계 핵심)

- BasePress.compress() 시그니처·내부 구조 먼저 파악 (쉽게: kvpress의 '압축 함수'가 어떻게 생겼는지 뜯어본다) → attention score 없이 외부 마스크로 진입하는 방식 확정
- 줄 점수 → 토큰 점수 변환(매핑 테이블 활용). 기존 press는 토큰별 attention 점수 기반이라 이 다리가 필요
- 레이어 조건 분기: module.layer_idx 접근해 초기 레이어는 통과, 이후부터 삭제 (쉽게: 지금 몇 번째 층인지 보고, 앞쪽 층은 안 건드리고 뒤 층부터 압축)
- 소형 테스트: '이 라인 버려' → 해당 토큰 KV가 (초기 레이어 이후) 전 레이어에서 사라지는지 확인

PoC 실패 백업

관문 실패 순간에 정하면 늦다. '막힌 뒤'가 아니라 '의심되는 순간' 병렬 착수한다.

순서	경로	판정
①	kvpress BasePress 상속 (본선)	C 담당
②	kvpress fork에 커스텀 press 추가	2단계 첫 주에 ① 시그니처 파악 실패 시 즉시 병렬 조사
③	transformers Cache 직접 서브클래싱	A가 6주차 이후 병렬 조사

통과 조건: PoC 수준에서 로그 신호 마스크 → (초기 레이어 이후) KV 삭제 동작 확인. 버퍼 +1~2주 (직접 Hook 전환 시).

3단계 (M4) 로그 신호 압축기 (LogPress) 구현 · 8~11주차 ★ 핵심

- kvpress BasePress 상속, 2단계 마스크 적용 LogPress 클래스 작성 [C 주담당]

- 가중 점수: 레벨 · 희귀도 · 시간 · 위치를 하나의 라인 점수로 합산 → 상위 keep
- 다중 신호 겹침으로 '드문 원인 vs 드문 무해' 구분 (드뭉+ERROR+장애시점 → 원인 / 드뭉+INFO+평상시 → 무해)
- 가중치 튜닝은 개발용 셋에서만 → 라벨 누수 방지 [D 감시]
- 입력 필터 + KV 직렬 파이프라인 동작 확인: 회장 피드백의 '두 개 구현' 제안이 여기서 실제 구현으로 합류

통과 조건: LogPress가 baseline과 동일 압축률에서 돌아가고, 개발셋에서 baseline보다 보존율 높은 경향 확인.

4단계 (M5) 1층 평가 — 압축률 대비 보존율 곡선 · 10~13주차 ★ 안전판

여기까지 완주하면 발표 가능한 결과가 나온다. → 발표 가능 최소선 [D 주담당]

- 홀드아웃 평가셋에서 LogPress vs baseline 4중 보존율 곡선(가로 압축률 50~90%, 세로 보존율)
- 총화 필수: (a) 전체 (b) 비-ERROR 원인 (c) 중간 위치 원인 — 프로젝트 동기가 된 하드 케이스에서의 우위 증명
- 통계 검정 (0단계에서 확보한 표본 규모로 검정력 확인 완료)

② Ablation 3중 (기본 설계 명시)

#	비교 대상	목적
①	기본 = 초기 레이어 통과 후 적용(FastKV식) vs 대조군 @ 레이어 0 부터 일괄 삭제 ① 레이어별 차등	설계 선택을 실증
②	입력 필터링 vs KV 압축	§2.1 도구 정당성 방어
③	필터 + KV 직렬 vs KV 단독	회장 피드백의 실험적 답변

Ablation ①의 의미: 단순 비교가 아니라 '왜 초기 레이어를 봐주는 설계를 택했는가'를 실증하는 실험이다. 기본을 명시하지 않으면 무엇을 주장하려는지 불분명해진다.

통과 조건: 하드 케이스 서브셋(비-ERROR 원인 · 중간 위치)에서 baseline 대비 통계적으로 유의한 보존율 우위 → 발표 가능 결과 확보.

5단계 (M6) 2층 평가 — RCA delta (검증 · 결정타) · 12~15주차 ★ 핵심

시작 전제: Nezha 전처리가 11주차 말에 완료되어 평가셋이 확보된 상태.

- 제약형 태스크: 자유서술 대신 '원인 후보 K개 중 선택' 또는 '원인 라인 지목' → floor 회피
- 같은 압축률에서 압축 방식만 바꿔 Llama-3.1-8B에 RCA → 정답률 delta 측정
- Nezha 평가(B가 11주차에 인계한 평가셋 투입) — Thunderbird는 이상 라벨이라 RCA 직접 검증 불가
- 2차 backbone Qwen2.5-7B 재현으로 일반화 확인(1단계 초기 세팅으로 빠름)

floor 백업 3단

단계	대응	결과
1차	제약형 태스크로 Llama-3.1-8B floor 회피	delta 측정 성공 시 종료
2차	DeepSeek-R1-Distill-Qwen-7B(추론 특화) 재시도	floor를 모델 능력으로 끌어올림
3차	5단계를 '방향성 관찰'로 격하	4단계(1층 총화 결과)를 핵심 성과로 프레이밍

통과 조건: delta가 측정 가능(floor 아님). 오르면 '보존→성능' 입증, 안 올라도 '정보가 아니라 모델 능력이 병목'이라는 발견.

6단계 (M7) 마무리 · 정리 · 13~16주차 ✓ 마무리

막판 몰림 완화를 위해 4단계 완주(13주차) 시점부터 롤링으로 병행 착수한다.

- Novelty 방어: '레벨 필터 대비 회귀도·위치 신호의 순증 마진'을 핵심 결과로 프레이밍
- 실패 케이스 분석
- 발표 / 논문 정리 [D 총괄 · 전원 분담]

4. 단계별 주담당 매트릭스 (R = 책임, S = 지원)

단계 / 마일스톤	A 인프라	B 데이터	C 매핑	D 리드
0 킬스위치 (M0-M1) ◆	R	R	S	R
1 Baseline + Nezha ◡	R	R	S	S
2 매핑 · PoC (M3) ◆	S	—	R	S
3 LogPress (M4)	—	S	R	S
4 1층 평가 (M5) ★	—	S	S	R
Nezha 완료 (M-N) ◆	—	R	—	R
5 2층 delta (M6)	—	R	S	R
6 마무리 (M7)	S	S	S	R

부하 곡선: A는 초반 무겁고 뒤 가벼움 · C는 중반 최대 · D는 처음·끝 · B는 전 구간 꾸준(두 선행 관문).

5. 관문 요약 · 버퍼 · 리스크 완충

3대 관문 (GO / NO-GO)

관문	주차	GO 조건	NO-GO 대응
M1 킬스위치	3주	하드케이스 표본 충분 + 신호 구별력	데이터 추가 / 신호 재설계 / 주제 재설정
M3 PoC	9주	마스크 → 전 레이어 KV 삭제 동작	fork 커스텀 → Cache 서브클래싱
M-N Nezha	11주	원인 라벨 라인 매핑 가능	5단계 방향성 관찰로 축소

관문 뒤 버퍼

관문	실패 시 발생 작업	권장 버퍼
M1 (3주)	데이터 추가 / 주제 재설정 회의	+1주 또는 즉시 판정
M3 PoC (9주)	직접 Hook 방식 전환(난이도↑)	+1~2주
M-N Nezha (11주)	5단계 축소 판단 → M6 계획 수정	+1주 (11주 확보로 완화)
M6 floor	추론 모델 재시도 / 1층 방어 전환	+1주

리스크 완충 구조: 4단계(1층 평가)까지만 완주해도 발표 가능한 결과가 나온다. 5단계(2층 RCA delta)는 floor 리스크가 있으니 '있으면 결정타, 없어도 1층으로 방어' 구조로 운영.

반증 조건

조건	대응
하드케이스 표본 과소 (수십 개)	BGL 병합 / 증화 완화 / 라인 단위 검정
신호 구별력 없음	신호 재설계 또는 주제 재설정
kvpress 혹은 PoC 실패	fork 커스텀 → Cache 직접 서브클래싱
필터만으로 GPU 메모리 목표 달성	KV 여지 없음 → 직렬 파이프라인 전제 재검토
RCA delta 미측정 (floor)	1층 결과로 방어 + '모델 능력 병목' 발견으로 프레이밍
baseline 대비 보존율 우위 없음	로그 규칙성 무용 → 신호 재설계 또는 주제 좁히기

6. 모델 · 도구 확정

항목	확정 내용 / 근거
주력 backbone	Llama-3.1-8B-Instruct — 128K · kvpress 예제 직접 명시 · KV 논문 업계 표준
2차 backbone	Qwen2.5-7B-Instruct 또는 Qwen3-8B — kvpress Qwen2/Qwen3 공식 지원
floor 백업 모델	DeepSeek-R1-Distill-Qwen-7B (추론 특화)
압축 프레임워크	NVIDIA kvpress — H2O · SnapKV · StreamingLLM · PyramidKV 등 30+ 내장
템플릿 파싱	Drain3 — pip 설치, 스트리밍 온라인 파싱
2층 RCA 태스크	제약형(원인 후보 K개 중 선택 / 원인 라인 지목) — floor 회피
△ 버전 주의	kvpress ↔ transformers 호환성 핀 고정 필수 (브레이킹 체인지 이력 있음)

7. 페어링 · 안전장치

- 초반 환경 셋업 페어링: A가 관문(전원 대기 리스크)이므로 0~1주 셋업만은 C나 D가 페어로 함께 → 지식 공유 + 병목 분산
- PoC 백업 사전 조사: C가 M3 진입과 동시에 A와 함께 '직접 Hook'(Cache 서브클래싱) 경로 병렬 조사
- 하드 케이스 조기 공유: B가 M1 시점에 비-ERROR-중간위치 원인 개수를 D에게 넘겨 통계 표본 판정
- Nezha 핸드오프: B의 Nezha를 D가 주 1회 점검하고, 11주 말 평가셋을 D에게 인계(M6 시작 전제)

한 문장 요약 3개 관문(3주 킬스위치 · 9주 PoC · 11주 Nezha)을 GO/NO-GO로 통과하며 임계 경로를 지키고, 13주 1층 완주로 안전판을 확보한 뒤 15주 2층 delta로 상방을 노린다. 핵심 설계는 '판단은 한번, 적용은 초기 레이어 이후'이며, 이것이 §2.1 도구 정당성과 정합한다.